

WIZUALIZACJA GRAFU PLANARNEGO

DOKUMENTACJA IMPLEMENTACYJNA

Mateusz Łoziński

343349

Krystain Ratajczyk

343375

Wydział Elektryczny PW, Informatyka Stosowana

10 Maja, 2026

Spis treści

1. Cel Projektu	3
2. Architektura Systemu	3
3. Struktura klas i pakietów	3
3.1. Pakiet <code>model</code>	3
3.2. Pakiet <code>view</code>	3
3.3. Pakiet <code>controller</code>	3
3.4. Pakiet <code>io</code>	4
4. Elementy Interfejsu Graficznego (GUI)	4
5. Kluczowe mechanizmy implementacyjne	4
5.1. Renderowanie Grafu (<code>GraphCanvas</code>)	4
5.2. Obsługa interakcji (Drag & Drop)	4
6. Obsługa Sytuacji Wyjątkowych i Błędów Wejścia	4
7. Zastosowane Wzorce Projektowe	5
8. Wykorzystane biblioteki	5

1. Cel Projektu

Głównym celem implementacyjnym projektu było zaprojektowanie wydajnej i przejrzystej aplikacji w języku Java (Swing) do wizualizacji struktury grafów. Podstawowym zbiorem danych wejściowych przetwarzanych przez aplikację są precyzyjne współrzędne geometryczne (X, Y) określające pozycję wierzchołków na płaszczyźnie, a także powiązania między nimi (krawędzie). Aplikacja jest jedynie nakładką graficzną na poprzednią część projektu, która jest silnikiem obliczeniowym służącym do wyznaczania współrzędnych wierzchołków grafu poprzez wybrany algorytm.

2. Architektura Systemu

Aplikacja została zaprojektowana w oparciu o wzorzec projektowy **MVC (Model-View-Controller)**, co zapewnia separację logiki biznesowej od interfejsu użytkownika i ułatwia testowanie oraz rozbudowę poszczególnych modułów.

- **Model:** Przechowuje strukturę danych grafu (wierzchołki, krawędzie) oraz logikę algorytmów.
- **View:** Odpowiada za renderowanie grafu na ekranie oraz budowę okien dialogowych.
- **Controller:** Zarządza przepływem danych, obsługuje zdarzenia myszy oraz komunikację z systemem plików.

3. Struktura klas i pakietów

Kod źródłowy został podzielony na odseparowane domeny funkcjonalne z zachowaniem zasad wzorca projektowego MVC. Logiczna struktura katalogów w projekcie przedstawia się następująco:

```
java_app/
|-- src/
|   |-- Main.java
|   |-- model/
|   |-- view/
|   |-- controller/
|   |-- io/
```

3.1. Pakiet `model`

Reprezentacja danych:

- **Graph.java:** Przechowuje zbiory wierzchołków i krawędzi.
- **Vertex.java:** Dane o współrzędnych i ID wierzchołka.
- **Edge.java:** Relacje między wierzchołkami wraz z wagami.

3.2. Pakiet `view`

Warstwa prezentacji:

- **MainFrame.java:** Konstrukcja okna i menu.
- **GraphCanvas.java:** Obszar rysowania (płótno), na którym renderowany jest graf.
- **ToolbarPanel.java:** Przyciski, suwaki i pola tekstowe.

3.3. Pakiet `controller`

Logika sterowania i obsługa zdarzeń:

- **GraphController.java**: Obsługuje zdarzenia myszy na płótnie (zaznaczanie, przesuwanie wierzchołków). Aktualizuje model i wywołuje odświeżenie widoku.
- **MenuController.java**: Reaguje na kliknięcia w menu (Otwórz, Zapisz), delegując zadania do modułu IO.

3.4. Pakiet io

Obsługa plików:

- **FileParser.java**: Parsowanie plików `.txt`.
- **BinaryManager.java**: Serializacja binarna obiektów.

4. Elementy Interfejsu Graficznego (GUI)

Aplikacja została zbudowana w oparciu o komponenty biblioteki `javax.swing`, zapewniając natywne środowisko okienkowe.

1. **MainFrame (Okno Główne)** Korzeń aplikacji używający `BorderLayout`.
 - Na górze okna znajduje się pasek nawigacyjny (`JMenuBar`), umożliwiający realizację operacji plikowych poprzez wbudowany komponent `JFileChooser` (wybór ścieżki na dysku).
 - Część centralną wypełnia komponent renderujący.
2. **GraphCanvas (Płótno Renderowania)** Klasa dziedzicząca po standardowym `JPanel`, służąca za dynamiczną powierzchnię malowania. Komponent ten jest sercem aplikacji wizualnej i reaguje na instrukcje repaintingu inicjowane przez kontrolery ruchu myszy.
3. **ControlPanel (Panel Boczny)** Osadzony po prawej stronie głównego okna. Składa się z elementów takich jak:
 - Lista rozwijana (`JComboBox`) – służąca wyborowi wariantu uruchamianego algorytmu.
 - Elementy stanowe (`JCheckBox` / `JRadioButton`) – do włączania i wyłączania metadanych renderowania (takich jak rysowanie identyfikatorów lub grubości krawędzi).

5. Kluczowe mechanizmy implementacyjne

5.1. Renderowanie Grafu (GraphCanvas)

Wizualizacja odbywa się poprzez iterację po liście krawędzi, a następnie wierzchołków. Wykorzystywany jest obiekt `Graphics2D` dla zapewnienia antyaliasingu.

5.2. Obsługa interakcji (Drag & Drop)

Zmiana współrzędnych wierzchołków realizowana jest przez `MouseListener` i `MouseMotionListener` które są połączone z `GraphCanvas`.

Przy kliknięciu (`mousePressed`) program sprawdza, czy kursor znajduje się w obrębie promienia wierzchołka.

Przy ruchu (`mouseDragged`) współrzędne wierzchołka są aktualizowane, a metoda `repaint()` wymusza odświeżenie widoku.

6. Obsługa Sytuacji Wyjątkowych i Błędów Wejścia

Aplikacja jest krytycznie zależna od poprawności wczytywanych współrzędnych wierzchołków grafu. W celu zachowania stabilności zaimplementowano szereg dedykowanych mechanizmów obronnych w oparciu o bloki `try-catch`.

1. **Walidacja Typów Liczbowych (`NumberFormatException`)** Podczas działania klasy **FileParser**, każdy odczytany token tekstowy odpowiadający współrzędnej przepuszczany jest przez funkcję weryfikującą **`Double.parseDouble()`**. Jeżeli podany znak nie reprezentuje liczby wymiernej, zgłaszany jest odpowiednio spreparowany wyjątek **`ParseException`** z informacją, która dokładnie linia pliku posiada błąd znakowy.
2. **Walidacja Spójności Grafu** Program sprawdza listę deklarowanych krawędzi. Jeżeli wejściowy strumień danych nakazuje przypiąć krawędź do początkowego lub końcowego wężła, którego współrzędne nie zostały nigdy wcześniej podane w pliku, logika modelu odrzuca błędny wpis logując naruszenie zasady integralności danych (tzw. **`Dangling Edge Exception`**). Wierzchołek, który nie występuje w puli **`Map<String, Vertex>`**, nie zostanie dołączony do grafu.
3. **Bezpieczeństwo Wejścia/Wyjścia (`IOException`)** Sytuacje takie jak: brak uprawnień dyskowych (np. w katalogu **`C:/Windows/`**), zabezpieczenie pliku przez zaporę lub fakt, że odczytywany dokument został nagle wykasowany podczas działania parsera, przechwytywane są na poziomie modułu I/O i skutkują zwróceniem łagodnego komunikatu w GUI, chroniąc tym samym cały system przed przymusowym wstrzymaniem.

7. Zastosowane Wzorce Projektowe

W celu ułatwienia późniejszej pielęgnacji kodu, zaimplementowano następujące wzorce:

- **Wzorzec Obserwator (`Observer`)**: Klasa **Graph** powiadamia zależne widoki (**GraphCanvas**, panele statystyk) o wszelkich zmianach w topologii, co eliminuje konieczność „ręcznego” wymuszania odświeżania okien.
- **Wzorzec Fabryka (`Factory`)**: Moduł wejścia/wyjścia posiada klasę **BinaryManager**, która na podstawie rozszerzenia pliku decyduje, czy użyć parsera tekstowego, czy deserializatora binarnego.

8. Wykorzystane biblioteki

- **Java Swing**: Standardowa biblioteka do budowy interfejsów graficznych.